NIGHTSHIFT · TECHNICAL GUIDE

How Nightshift works, component by component

A plain-language but complete explanation of every backend part of Nightshift: what it does, how it does it, the exact formulas, and how to explain it to someone else.

IN ONE PARAGRAPH

Tokenized ETFs (blockchain tokens that track funds like SPY or a Bitcoin ETF) trade 24/7, but their reference price only updates while Wall Street is open. Overnight and on weekends, normal price feeds keep repeating Friday's close. Nightshift computes a **fair value** for these tokens around the clock from markets that are still trading, then builds four products on that one price: a **lending vault**, a **loan guardian**, a **privacy proof (Cloak)**, and a tamper-proof **record on Algorand (Ledger)**.

Live site: **https://nightshift.buzz** · Version: October 2026 testnet build

Code: Next.js (TypeScript) app with server API routes, Solidity smart contracts, a Circom zero-knowledge circuit.

Contents

1. **The big picture:** the problem, the five modules, the architecture
2. **Market data layer:** where prices come from
3. **The Oracle engine:** how one fair value is computed
4. **Learned weights:** how the Oracle tunes itself from its own track record
5. **News agent (Claude):** the optional AI signal
6. **Shift report:** proving the Oracle works
7. **Public API:** how others read the data
8. **Smart contracts:** tokens, on-chain oracle, Morpho adapter, Vault
9. **The Guardian:** loan protection, on-chain and in the browser
10. **The keeper and the faucet:** the server jobs that run the chain side
11. **The Cloak:** zero-knowledge proof of solvency
12. **The Ledger:** predictions stamped on Algorand
13. **Infrastructure:** hosting, schedules, secrets, domain
14. **Status:** what is live, what is testnet, what is next
15. **Explaining it:** short scripts, likely questions and honest answers
16. **Glossary**

> **HOW TO READ THIS**
>
> Each component section has the same shape: **what it is**, **why it exists**, **how it works** (with the real numbers and formulas), **where it lives in the code**, and a short **"how to explain it"** line you can say out loud. Status badges: `LIVE` running with real data, `TESTNET` real blockchain code with test money, `PAPER` simulated in the browser, `READY / OFF` built but waiting for a key or funding.

1. The big picture

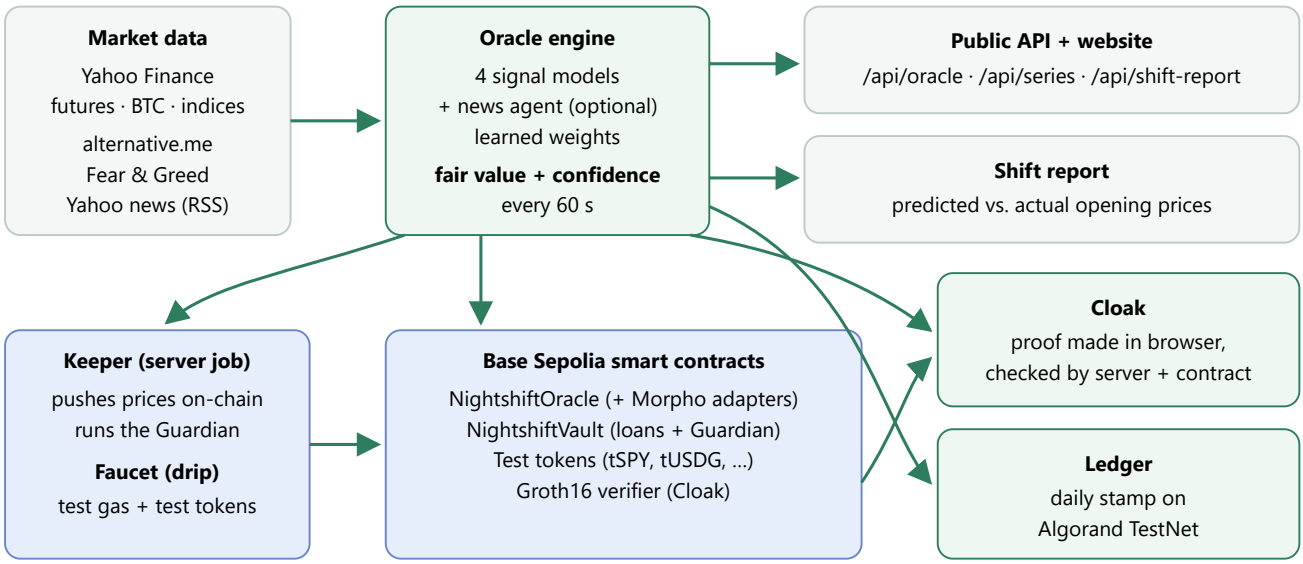
The problem

- **Prices go stale.** Tokens trade 24/7, but traditional price feeds stop at Friday 4:00 PM New York time and repeat that number all weekend.
- **Monday gaps.** Weekend news reaches the price all at once at Monday's 9:30 AM open. A loan that looked safe on a frozen price can be instantly underwater.
- **Loans are visible.** On a public blockchain anyone can see each loan's size and exact liquidation price, so automated bots know precisely where to push the price to profit from a liquidation.

The five modules

Module	Job, in one sentence	Status
Oracle	Computes a fair value and a confidence score for SPY, QQQ, GLD and IBIT tokens every minute, even when markets are closed.	LIVE
Vault	Lets users borrow USDG (a dollar stablecoin) against ETF tokens, with limits set from the Oracle's price.	TESTNET + PAPER
Guardian	Watches every loan and, if the borrower opted in, sells just enough collateral to keep it safe before it can be liquidated.	TESTNET + PAPER
Cloak	Lets a borrower prove a loan is safely backed without revealing its size or liquidation price, using a zero-knowledge proof.	LIVE
Ledger	Writes each pre-open prediction to the Algorand blockchain so the track record can never be edited later.	READY

Architecture: how the parts connect



Everything flows from one place: the Oracle's fair value. The website, the API, the shift report, the on-chain oracle, the Vault limits, the Guardian, the Cloak's proof checks and the Ledger stamps all read that same number.

> **HOW TO EXPLAIN IT**
>
> *"When Wall Street closes, these tokens keep trading but their price feed freezes. Nightshift keeps pricing them from markets that are still open, and then uses that live price to run safer loans, protect borrowers overnight, let them prove solvency privately, and publish a track record nobody can edit."*

2. Market data layer LIVE

What it is: the code that fetches raw market prices and turns them into clean time series the Oracle can use. `lib/data.ts`, `lib/time.ts`.

Why it exists: the Oracle can only be as good as its inputs. It needs two kinds of data: the ETF's own official prices (to know the last real close) and prices of things that keep trading overnight (to know what has changed since).

Sources

Data	Symbol	Used for	Refresh
ETF daily prices	`SPY, QQQ, GLD, IBIT`	Official opens and closes (the "frozen" price)	10 min cache
S&P 500 futures	`ES=F`	Overnight anchor for SPY	2 min cache
Nasdaq-100 futures	`NQ=F`	Overnight anchor for QQQ	2 min
Gold futures	`GC=F`	Overnight anchor for GLD	2 min
Bitcoin spot	`BTC-USD`	24/7 anchor for IBIT (Bitcoin ETF)	2 min
Nikkei, DAX, FTSE	`^N225, ^GDAXI, ^FTSE`	Asian and European moves while the US sleeps	2 min
Crypto Fear & Greed index	alternative.me	Market mood (sentiment signal)	30 min

All sources are free and need no API key. Prices come from Yahoo Finance's public chart endpoint in hourly bars (and 5-minute bars for the live chart).

How it works

1. **Fetch and clean.** Each request has a 6-second timeout. Bars with missing values are dropped; a series with fewer than 5 bars counts as a failure.
2. **Know the trading calendar.** `lib/time.ts` converts timestamps to New York time and knows the regular session (Mon–Fri, 09:30–16:00 ET). It labels the current phase: regular session, overnight, pre-dawn (Asia/Europe hours) or weekend. (Market holidays are not yet modelled.)
3. **Turn daily bars into sessions.** Each trading day becomes a record with its exact 09:30 open time, 16:00 close time, opening price and closing price.
4. **"Price at time T" without cheating.** The helper `priceAt(bars, T)` returns the price at time T using only bars that started at or before T. This guarantees that when the system replays history (the shift report), it never accidentally uses information from the future.
5. **Never break.** If a source fails (for example Yahoo blocks a server), that asset switches to a deterministic **simulated** series and everything downstream is labelled `"source": "simulated"`. The website shows a visible "simulated" badge. The math is identical, so the product keeps working and stays honest.

HOW TO EXPLAIN IT

"We watch markets that never fully close (index futures, Bitcoin, Asian and European stock markets, and a sentiment index) and line them up against each ETF's last official close."

3. The Oracle engine LIVE

What it is: the core of Nightshift. Every minute it produces, for each token, a **fair value** (what the price should be right now) and a **confidence score** from 0 to 100. `lib/oracle.ts`, `lib/config.ts`.

Why it exists: lenders need a realistic price at 3 AM on a Sunday, not Friday's close.

Step 1: is Wall Street open?

If the regular session is open, the Oracle simply uses the ETF's real market price, with confidence 99. Everything below applies when the market is **closed**.

Step 2: four signal models each make an estimate

Let **C** be the last official close and **a** the percentage move of the anchor (futures or Bitcoin) since that close.

Signal	Formula	Idea
Anchor	`C × (1 + a)`	If S&P futures are up 0.5% since the close, SPY is probably up about 0.5% too. The strongest signal.
Global	`C × (1 + s × g)`	**g** = average move of Nikkei, DAX and FTSE since the close; **s** = how much this asset follows them (SPY 0.5, QQQ 0.55, GLD 0.1, IBIT 0.3).
Momentum	`Anchor × (1 + 0.25 × m)`	**m** = the anchor's move over the last 3 hours. Captures a trend that is still developing.
Sentiment	`Anchor × (1 + t × ΔFG / 50)`	**ΔFG** = day-over-day change in Fear & Greed; **t** = tilt (SPY 0.2%, QQQ 0.3%, GLD −0.1% because gold rises on fear, IBIT 0.8%).
News (optional)	`Anchor × (1 + n × score)`	Claude's headline score from −1 to +1; **n** = SPY 0.4%, QQQ 0.5%, GLD 0.3%, IBIT 1%. See section 5.

Step 3: blend them

```
fair value = Σ (weight_i × estimate_i)      weights are normalised to sum to 1
```

Starting weights: anchor 60%, global 20%, momentum 10%, sentiment 10%, news 10% (when enabled). Anchor, global and momentum are re-tuned automatically from the track record (section 4). If the sentiment feed is down, its weight drops to 0.

Step 4: confidence

```
confidence = clamp( 98 − 0.3 × hoursSinceClose − 0.5 × disagreementBps − (10 if simulated), 35, 98
)
```

- **Time penalty:** the longer markets have been closed, the less sure we are (−0.3 points per hour).
- **Disagreement penalty:** how far the signals are from each other, measured as a weighted standard deviation in basis points (1 bp = 0.01%).
- **Simulated data:** −10 points. Confidence never goes below 35 or above 98.

WORKED EXAMPLE: SPY ON A SATURDAY (REAL NUMBERS)

Last close $778.57. S&P futures slightly down, Asia/Europe closed, Fear & Greed up 5 points.
Anchor $778.45 (weight 59%), Global $778.57 (11%), Momentum $778.45 (20%), Sentiment $778.35 (10%).
Fair value $778.45, about 0.02% below the frozen close. **Confidence 85**, mostly because ~38 hours have passed since the close.

HONEST NOTE

This is a transparent statistical model, not a neural network. That is deliberate: lenders and auditors can check every number. The "AI" parts are the self-tuning weights and the optional Claude news agent.

HOW TO EXPLAIN IT

"Four simple, explainable models each guess where the price should be. We blend them, weighted by how accurate each has been recently, and attach a confidence score that drops the longer the market stays closed."

4. Learned weights LIVE

What it is: the mechanism that re-tunes how much the Oracle trusts each signal, based on how well each one predicted recent opening prices. `lib/weights.ts`.

How it works

1. For each of the last ~20 trading days, it pretends it is 09:30 New York time and asks each signal (anchor, global, momentum) to predict the open, using **only data available before the open**.
2. It compares each prediction with the real opening price and records the squared percentage error.
3. Signals with smaller average error get more weight: weight ∝ 1 / (mean squared error).
4. To avoid overreacting to one unusual week, the result is blended **50/50 with the starting weights**.
5. Safety rules: it only adjusts when it has at least 5 days of **live** data. Sentiment and news keep fixed weights (there isn't enough reliable history to fit them).

Result today (SPY): anchor 59%, global 11%, momentum 20%, sentiment 10%. Momentum earned more weight and the global signal less than the starting guess. The website notes "Weights re-fit from the last 20 opens" under each price card.

> HOW TO EXPLAIN IT
>
> *"Every day the Oracle grades its own signals against the real opening bell and shifts trust toward whichever has been most accurate, carefully, so one strange week can't swing it."*

5. News agent (Claude) READY / OFF

What it is: an optional fifth signal. An AI model (Claude, by Anthropic) reads the latest headlines for each ETF and scores their likely effect on the next open. `lib/news.ts`.

How it works

1. Fetches up to 8 recent headlines per ticker from Yahoo Finance's public RSS feeds.
2. Sends them to Claude with clear instructions: score from −1 (strongly negative) to +1 (strongly positive), near 0 if mixed or irrelevant, plus a one-sentence reason. Headlines are treated as data, never as instructions.
3. The answer is forced into a strict JSON shape (structured outputs), so it can be parsed reliably.
4. The result is **cached for one hour**, so it runs at most about 24 times a day.
5. The score becomes the "news" estimate in the Oracle (section 3) with a 10% weight, and its reason is shown on the price card.

Switching it on: add an `ANTHROPIC_API_KEY` in Vercel. It uses Claude Opus 5.5 at low effort by default (roughly $5–15 a month at this frequency); the model can be changed with `NEWS_MODEL`. Without a key, the Oracle runs on its four market signals and says the news agent is off.

HOW TO EXPLAIN IT

"An AI reads the weekend's headlines for each fund and nudges the price a little in the direction the news points. It's capped and weighted, so it informs the price but can't dominate it."

6. Shift report LIVE

What it is: the scorecard that proves the Oracle works. `lib/backtest.ts`, `/api/shift-report`.

How it works

1. For each of the last 10 trading days, take the previous close and the anchor's move from that close to 09:30 the next morning.
2. Predict the open: `previous close × (1 + anchor move)`, using only pre-open data.
3. Compare with the **actual** open, and with what a traditional frozen feed would have said (the previous close).
4. Report the average error of each (in basis points), the error reduction, and how many mornings Nightshift was closer.

Current results (live data, October 2026)

Token	Nightshift avg. error	Frozen feed avg. error	Error reduction	Mornings won
SPY	5.9 bps (0.06%)	33.9 bps (0.34%)	83%	9 of 10
QQQ	9.4 bps	54.8 bps	83%	9 of 10
GLD	23.5 bps	100.1 bps	77%	9 of 10
IBIT	17.9 bps	140.2 bps	87%	9 of 10

Figures change daily as new sessions enter the window; the site always shows the current ones.

> **HOW TO EXPLAIN IT**
>
> *"Every morning we grade our overnight price against the real opening bell. Over the last ten opens we were about five times more accurate than the frozen price lenders use today."*

7. Public API LIVE

What it is: simple web addresses that return JSON, so wallets, dashboards, bots or other protocols can read Nightshift's data. No key needed; cross-origin requests are allowed.

Address	Returns	Cache
`GET /api/oracle`	Market status and all four quotes: fair value, confidence, drift, every signal's estimate and weight	60 s
`GET /api/oracle/SPY`	One asset	60 s
`GET /api/series?token=SPY&range=12H`	Fair-value history on 5-minute data (1H, 4H, 12H or 24H)	2 min
`GET /api/shift-report`	Predicted vs. actual opens for the last 10 sessions	10 min
`GET /api/ledger`	Predictions stamped on Algorand, with real outcomes	—
`GET /api/chain`	On-chain status: prices stored in the contract, number of positions	30 s
`POST /api/zk/verify`	Checks a Cloak proof (section 11)	—

"Cache" means the hosting network serves a recent copy for that long, which keeps the site fast and protects the free data sources from traffic spikes.

8. Smart contracts TESTNET

What they are: programs that run on a blockchain (Base Sepolia, Coinbase's public test network) and hold the real rules for loans. Written in Solidity, folder `contracts/`. Test assets have no monetary value, which is exactly what a demo needs.

> **STATUS**
> Built, compiled and passing 15 of 15 automated tests on a local blockchain, plus a full browser test. Deployment to Base Sepolia is waiting only for the deployer wallet to be funded with free test ETH from a faucet.

8.1 TestToken (×5)

Simple ERC-20 tokens: **tUSDG** (the loan currency) and **tSPY, tQQQ, tGLD, tIBIT** (collateral). Anyone can claim a small amount once an hour through `faucet()` (e.g. 10 tSPY, 200 tIBIT); the deployer can mint more to seed liquidity.

8.2 NightshiftOracle

Stores the Oracle's fair value on-chain so contracts can use it. For each token it keeps **price** (18 decimals), **confidence** (0–100) and **updatedAt**. Only authorised "updaters" (the keeper wallet) can call `setPrices()`; anyone can read `getPrice()`.

8.3 MorphoOracleAdapter (×4)

Morpho is a large, widely used lending protocol. Its markets read prices through a standard function, `price()`, scaled by 10^{36}. The adapter translates our on-chain oracle into exactly that format, one adapter per token. This means Nightshift's price can plug straight into real Morpho markets later.

8.4 NightshiftVault: the lending contract

Function	What it does
`deposit(token, amount)`	Moves collateral from the user into the vault.
`borrow(token, amount)`	Sends tUSDG to the user, only if total debt stays within the borrow limit.
`repay(token, amount)`	Pays back debt.
`withdraw(token, amount)`	Returns collateral, only if the health factor stays at or above 1.10.
`setRule(token, mode, trigger, target)`	The borrower's Guardian settings (section 9).
`protect(user, token)`	The Guardian action (section 9). Anyone may call it, but it only acts within the borrower's own rule.
`liquidate(user, token, amount)`	If health factor < 1, a liquidator repays debt and receives collateral plus a bonus.

The key numbers

- **LLTV (liquidation loan-to-value)**: the maximum debt as a share of collateral value before liquidation. 77% for SPY, QQQ and GLD; 62.5% for IBIT (Bitcoin is more volatile). These match Morpho's standard tiers.
- **Health factor** = collateral value × LLTV ÷ debt. Above 1 is safe; below 1 can be liquidated.
- **Borrow limit** = collateral value × LLTV × (1 − (100 − confidence) ÷ 200). Example: confidence 86 → limit cut by 7%. Lower confidence means more caution.
- **Liquidation bonus (LIF)** = min(1.15, 1 ÷ (0.3 × LLTV + 0.7)), Morpho's formula. At 77% LLTV it is about 1.074, so a liquidator who repays $100 of debt receives $107.40 of collateral.
- **Stale-price protection**: the vault refuses to use a price older than 7 days.
- **Simplifications for the demo**: no interest is charged, and the vault itself holds the tUSDG liquidity (50 million test tokens).

8.5 Groth16Verifier

Generated automatically from the Cloak's zero-knowledge circuit. It checks a Cloak proof on-chain with one function, `verifyProof(...)`. Reading it is free, so the website can ask the blockchain "is this proof valid?" without a wallet (section 11).

Tests that pass

Faucet and cooldown · borrow limit equals value × LLTV × confidence haircut · over-limit borrow is refused · Morpho adapter scaling · Guardian refuses before opt-in · Guardian restores health 1.04 → exactly 1.40 after an 8% price drop · only updaters can set prices · health < 1 after a 40% crash · liquidator receives debt × 1.074 in collateral · on-chain verifier accepts a real proof and rejects a forged price · position registry.

9. The Guardian TESTNET PAPER

What it is: automatic loan protection. It watches each loan and acts **before** the loan can be liquidated, so the borrower avoids the liquidation penalty and a forced sale at the worst moment.

The borrower chooses a rule per loan

Mode	What happens when health drops below the trigger
Off	Nothing (normal liquidation rules still apply).
Alert	A warning (in the paper version: on-screen toast and optional browser notification, at most every 30 minutes).
Auto-protect	Sells just enough collateral to repay debt and bring health back to the chosen target.

Typical settings: trigger 1.15, target 1.40. The target must be at least 0.05 above the trigger, so the Guardian doesn't fire again immediately.

The exact amount to sell

With P = price, u = collateral units, D = debt, L = LLTV, T = target, s = 0.5% slippage (selling slightly below the price, like a real thin market):

```
k = (T × D − P × u × L) ÷ (P × (T × (1 − s) − L))
```

Selling k units at P × (1 − s) and repaying the proceeds leaves the loan at exactly health T. This is solved in one step, not by trial and error.

> **REAL TEST RESULT**
> A loan with 5 tSPY and $2,700 debt had health 1.11 (below the 1.15 trigger). The keeper ran the on-chain Guardian: it sold 1.61 tSPY, repaid $1,250, and health became exactly **1.40**. No liquidation, no penalty.

Two implementations

- **On-chain (testnet):** rules are stored in the Vault contract (`setRule`). The keeper (section 10) checks every opted-in loan on each run and calls `protect()`. Because the contract enforces the borrower's own rule, the keeper cannot do anything the borrower didn't allow, and anyone could run a keeper.
- **Paper (browser):** the same math runs in the visitor's browser on every Oracle update (`lib/paper.ts`), for trying it without a wallet. It also offers a "shock test" (what happens to my loans if prices gap −8%?) and a step-by-step weekend walkthrough.

> **HOW TO EXPLAIN IT**
> *"You set a safety line on your loan. If the overnight price pushes you below it, the Guardian sells just enough collateral to get you back to safety, before any liquidation bot can touch you."*

10. The keeper and the faucet TESTNET

Keeper (`/api/keeper`, `lib/chain/keeper.ts`)

A server job, protected by a secret, that does two things each time it runs:

1. **Pushes prices on-chain**: for each token, if the Oracle's fair value moved more than 0.1% from the stored value, or the stored value is older than 30 minutes, it writes the new price and confidence to the NightshiftOracle contract (one transaction for all changed tokens). This saves gas when nothing moved.
2. **Runs the Guardian**: reads every position from the Vault, and for each opted-in loan whose health is below its trigger (but still above 1), calls `protect()`.

Schedule: once a day by Vercel's free cron as a fallback; every 5 minutes by a free external scheduler (cron-job.org) once that is set up.

Faucet / drip (`/api/drip`)

Lets anyone try the testnet Vault with only a wallet. On request it sends a little test ETH for gas (0.0004 ETH, only if the wallet has less than 0.00015) and test tokens (10 tSPY, 10 tQQQ, 20 tGLD, 200 tIBIT, only if the wallet holds less). Limits: once per 10 minutes per address, and it stops if the keeper wallet's balance falls below a reserve.

11. The Cloak: zero-knowledge proof of solvency LIVE

What it is: a way for a borrower to prove "my loan is safely backed at today's price" without revealing how much they borrowed, how much collateral they have, or where their liquidation price is.

Why it exists: on public blockchains, visible loans are targets. Bots can see exactly how far the price needs to move to liquidate someone. Hiding the numbers removes the target; the proof keeps lenders confident.

What "zero-knowledge proof" means

A short cryptographic certificate that a statement is true, which reveals nothing beyond the fact that it is true. Here the statement is a little math problem about secret numbers.

The circuit (the statement being proven)

File `zk/solvency.circom`, written in Circom, 918 constraints.

Input	Visibility
Oracle price (in cents), market LLTV	Public
Collateral amount, debt amount, a random secret "salt"	**Private** (never leaves the browser)
Commitment (a fingerprint of the position)	Public output

The circuit proves both of these at once:

1. `collateral × price × LLTV > debt`, i.e. the health factor is above 1.
2. `commitment = Poseidon(collateral, debt, salt)`. Poseidon is a hash function designed for zero-knowledge circuits; the salt stops anyone guessing the numbers by trial.

Range checks (64, 64, 40 and 14 bits) stop anyone cheating with giant numbers that wrap around the arithmetic.

The proving system

- **Groth16 on the BN254 curve**, via the snarkjs library. It is the same proof system Zcash made production-grade.
- Proofs are about 720 bytes; proving takes about 0.3–0.8 seconds in the browser.
- **Trusted setup** (a one-time key ceremony Groth16 needs): phase 1 is the public PSE "Perpetual Powers of Tau" ceremony with 80 independent contributors; phase 2 is a single demo contribution. A production launch needs a multi-party phase 2.

The flow

1. The browser generates the proof from the private numbers.
2. The server (`/api/zk/verify`) checks three things: (a) the proof is mathematically valid, (b) the price inside it is within 0.5% of the **live Oracle** (otherwise someone could prove "safe" at a made-up price), and (c) the LLTV matches the market. All three must pass.
3. Optionally, the deployed **Groth16Verifier contract** checks it on-chain (free to read).

Tested: an honest proof is accepted; changing the price after proving is rejected; a real proof submitted for the wrong token fails the price check; an under-collateralised loan cannot produce a proof at all. Proofs can be copied and checked by anyone with the "Verify a shared proof" tool.

HONEST NOTE

Today the Cloak proves solvency of a position; the position itself is still visible on-chain. Fully hidden ("shielded") positions are a larger research project and need a security audit.

HOW TO EXPLAIN IT

"It's like showing a bouncer a sealed envelope that the bouncer's scanner can confirm says 'over 18', without ever seeing your birthday."

12. The Ledger: predictions on Algorand READY

What it is: a permanent public record of the Oracle's predictions, written before each market open, so the track record cannot be edited or cherry-picked later. `lib/algorand.ts`, `/api/commit`, `/api/ledger`.

Why Algorand: fast, essentially free, final in about 3 seconds, and independent of the network where lending happens. If one chain has problems, the record survives on the other.

How it works

1. **Every weekday at 12:00 UTC** (before the 09:30 New York open), a scheduled job calls `/api/commit` (protected by a secret).
2. It takes the current Oracle snapshot: target session date, timestamp, each token's predicted price and confidence.
3. It computes a SHA-256 fingerprint of that data.
4. It sends a **0-ALGO payment to itself** on Algorand TestNet, with the prediction and fingerprint in the transaction's note field (prefix `nightshift:v1:`). Cost: 0.001 test ALGO.
5. `/api/ledger` reads these transactions back from Algorand's public indexer and pairs each prediction with the real opening price once it exists. **The blockchain is the database**; there is no other storage.

Status: the account and secrets are configured. It starts stamping as soon as the account is funded with free TestNet ALGO from a faucet.

> **HOW TO EXPLAIN IT**
> *"Before every opening bell we publish our prediction to a public blockchain. Anyone can check afterwards; we couldn't change it if we wanted to."*

13. Infrastructure

Piece	Detail
App	One Next.js 15 (TypeScript) application: website and server API routes together.
Hosting	Vercel (free Hobby tier), project `nightshift3`, served at **nightshift.buzz** (GoDaddy DNS → Vercel, automatic HTTPS; www redirects to the main domain).
Scheduled jobs	Vercel cron: Ledger commit weekdays 12:00 UTC; keeper once daily 12:30 UTC. External 5-minute keeper via cron-job.org (planned).
Secrets	Stored encrypted in Vercel and locally in `.env.local` (never uploaded or committed): keeper wallet key, cron secret, Algorand account phrase, optional Claude API key. All are testnet-only.
Blockchains	Base Sepolia (lending contracts, ZK verifier) and Algorand TestNet (Ledger). Public RPC endpoints, no paid provider.
Browser state	The paper-trading account lives in each visitor's own browser (localStorage), so every visitor gets their own starting wallet.
Tooling	solc (contract compiler), viem (blockchain calls), Hardhat (local test chain), Circom + snarkjs (ZK), algosdk (Algorand), Anthropic SDK (news agent).

Running cost today: $0. Only the optional news agent costs money (a few dollars a month).

14. Status: what's real, what's next

Real and live today	Built, waiting only for free setup	Next (needs funding / time)
• Oracle with live market data • Learned signal weights • Shift report on real opens • Public API • Cloak zero-knowledge proofs (browser + server) • Paper Vault and Guardian • Custom domain with HTTPS	• Smart contracts on Base Sepolia (fund deployer with test ETH) • On-chain Guardian + keeper (schedule every 5 min) • On-chain proof verification • Algorand Ledger (fund with test ALGO) • Claude news agent (add API key)	• Markets on Morpho itself, then Robinhood Chain • Licensed market data feeds • Multi-party trusted setup; shielded positions • Alerts by email / Telegram • Security audit and legal review

Risks and how they're handled

Risk	Mitigation
A data source fails or blocks us	Automatic labelled fallback; confidence drops; production would use licensed feeds.
Oracle is wrong at the open	Confidence-based borrow haircut; the Guardian acts before liquidation; the shift report makes errors public.
Someone fakes a solvency proof	Cryptographic check plus price and LLTV checks against the live Oracle; the circuit refuses unsafe inputs.
Keeper misbehaves	The contract only lets it act within each borrower's own rule; only authorised updaters can set prices; stale prices are refused after 7 days.
Predictions edited after the fact	Stamped on Algorand before the open.

15. Explaining it to other people

The 30-second version

"Tokenized ETFs trade around the clock, but their price feeds freeze when Wall Street closes. Nightshift prices them overnight from futures, Bitcoin, world markets and sentiment, and we've been about five times more accurate than the frozen price at predicting the open. On top of that price we've built a lending vault, a guardian that protects loans overnight, private proofs of solvency, and a public track record on Algorand. It's live on testnet at nightshift.buzz."

For a technical audience (2 minutes)

"The Oracle is an ensemble of four transparent signals: anchor futures or BTC, Asia/Europe indices, a three-hour momentum term and Fear & Greed, plus an optional Claude news score. Weights are re-fit from inverse mean squared error on the last ~20 opens, blended 50/50 with priors. Confidence decays with time since close and signal dispersion. A keeper pushes prices to an on-chain oracle with a Morpho IOracle adapter. The Vault uses Morpho Blue's LLTV and LIF math with confidence-haircut borrow limits, and borrowers can opt into an on-chain Guardian whose deleveraging amount is solved in closed form. The Cloak is a 918-constraint Groth16 circuit proving HF > 1 against a Poseidon commitment, verified by the server against the live Oracle and by a Solidity verifier. Predictions are stamped pre-open in Algorand transaction notes."

Questions you'll get, and honest answers

Question	Answer
Is this AI?	The core is a transparent statistical model that tunes its own weights from its track record. The optional news agent uses Claude. We avoid black boxes on purpose: lenders need to audit prices.
Is real money involved?	No. Testnet tokens and a paper mode. Real funds come after an audit.
Do you use Morpho / Aave / Zcash / Monero?	Vault follows Morpho Blue's rules and exposes a Morpho-compatible oracle; live Morpho markets are next. Aave is a candidate second venue. The Cloak uses the same proof system as Zcash. Monero is a design reference only. None are partners.
How do we know the Oracle is accurate?	The shift report grades it every morning on real opens, and the Ledger will make those predictions tamper-proof.
What if the market gaps hugely on Monday?	Confidence is lower over weekends, so borrow limits are tighter; the Guardian de-risks on Sunday night as the Oracle sees the move coming; liquidation follows Morpho's standard rules if all else fails.
Are positions private today?	Solvency can be proven privately today; fully hidden positions are on the roadmap.
Why Algorand and Base?	Separation: the record lives apart from where lending happens. Base (an Ethereum layer-2) is where Morpho and EVM tooling live; Robinhood Chain is the target network for tokenized ETFs.
What does it cost to run?	$0 today on free tiers; only the news agent has a small API cost.

16. Glossary

Basis point (bp)	0.01%. 34 bps = 0.34%.
Collateral	Assets deposited to secure a loan.
Confidence	Nightshift's 0–100 score for how reliable the current fair value is.
Fair value	Nightshift's best estimate of what a token's price should be right now.
Frozen / stale feed	A traditional price feed that keeps repeating the last official close while markets are shut.
Health factor (HF)	Collateral value × LLTV ÷ debt. Below 1 means the loan can be liquidated.
Keeper	A server job that performs routine on-chain actions (updating prices, running the Guardian).
LIF	Liquidation incentive factor: the bonus a liquidator receives (≈7.4% at 77% LLTV).
Liquidation	Forced repayment of an unsafe loan by a third party, who takes collateral at a discount.
LLTV	Liquidation loan-to-value: maximum debt as a share of collateral value (77% or 62.5% here).
Morpho	A widely used decentralised lending protocol whose rules Nightshift's Vault follows.
Oracle	A service that brings outside data (prices) to blockchains.
Poseidon	A hash function designed to be efficient inside zero-knowledge circuits.
Smart contract	A program that runs on a blockchain and enforces its rules automatically.
Testnet	A public blockchain for testing, using tokens with no real value (Base Sepolia, Algorand TestNet).
Tokenized ETF	A blockchain token that tracks an exchange-traded fund such as SPY.
Trusted setup	A one-time key ceremony some proof systems need; safe as long as one participant was honest.
USDG	A US-dollar stablecoin; the loan currency (tUSDG on testnet).
Zero-knowledge proof	A cryptographic proof that a statement is true which reveals nothing else.

Oracle values are data, not investment advice. Stock tokens are not available to US persons. Third-party names describe technical relationships only and do not imply endorsement or partnership.